

# 选课管理系统 — 项目说明文档

## 目录

- [1. 项目概述](#)
- [2. 快速启动](#)
- [3. 项目结构](#)
- [4. 技术栈](#)
- [5. 数据库设计](#)
- [6. 通用开发模式](#)
- [7. 系统管理员](#)
- [8. 学院管理员](#)
- [9. 教师](#)
- [10. 学生](#)
- [11. 测试账号](#)
- [12. 常见问题](#)

## 1. 项目概述

本系统是一个完整的 Web 端选课管理系统，支持四种角色协作完成课程发布、选课、抽签、成绩录入等业务流程。

**业务流程：** 管理员创建学院和师生 → 教师发布课程 → 学生选课 → 学院管理员抽签 → 中签学生上课 → 教师录入成绩 → 师生消息互动

**四种角色：**

| 角色                    | 职责                        |
|-----------------------|---------------------------|
| 系统管理员 (admin)         | 全局管理：创建学院、管理所有师生、课程、成绩    |
| 学院管理员 (college_admin) | 学院内管理：管理本学院师生、课程、抽签       |
| 教师 (teacher)          | 课程管理：编辑课程信息、上传课件、录入成绩、发消息 |
| 学生 (student)          | 选课上课：浏览课程、选课、查成绩、下载课件、发消息 |

## 2. 快速启动

```
# 1. 安装依赖
cd backend
pip install -r requirements.txt

# 2. 启动后端
python app.py
# 访问 http://localhost:5000
```

首次启动会自动创建数据库并填充测试数据。如需重置数据库，删除 `database/course_selection.db` 后重启即可。

### 3. 项目结构

```
course-selection-system/
├─ database/
│   └─ schema.sql          # 建表SQL + 测试数据（启动时自动执行）
├─ backend/
│   ├── app.py             # Flask 入口，注册蓝图，静态文件路由
│   ├── config.py          # 配置（密钥、路径、分页大小等）
│   ├── database.py        # SQLite 连接管理（WAL模式，外键强制）
│   ├── models/            # 数据模型层
│   │   ├── user.py        # 用户模型
│   │   ├── course.py      # 课程模型
│   │   ├── selection.py   # 选课记录模型（含抽签算法）
│   │   ├── grade.py       # 成绩模型（含统计分析）
│   │   ├── message.py     # 消息模型
│   │   └─ course_material.py # 课件模型
│   ├── routes/            # 路由层（按角色分文件）
│   │   ├── auth.py        # 认证接口
│   │   ├── admin.py       # 系统管理员接口
│   │   ├── college_admin.py # 学院管理员接口
│   │   ├── teacher.py     # 教师接口
│   │   └─ student.py      # 学生接口
│   └─ uploads/            # 课件文件存储目录
├─ frontend/
│   ├── index.html         # 登录页
│   ├── css/style.css      # 全局样式
│   ├── js/api.js          # API 通信层（前端统一入口）
│   └─ pages/
│       ├── admin.html     # 系统管理员界面
│       ├── college_admin.html # 学院管理员界面
│       ├── teacher.html   # 教师界面
│       └─ student.html    # 学生界面
```

### 4. 技术栈

| 层    | 技术                                     |
|------|----------------------------------------|
| 后端框架 | Python Flask                           |
| 数据库  | SQLite（WAL 模式，外键强制开启）                  |
| 前端   | 原生 JavaScript + Bootstrap 5 + Chart.js |
| 认证   | Flask Session（Cookie 签名）               |

## 5. 数据库设计

### 5.1 表结构

colleges (学院表)

| 字段          | 类型                  | 说明   |
|-------------|---------------------|------|
| id          | INTEGER PK          | 自增主键 |
| name        | VARCHAR(100) UNIQUE | 学院名称 |
| description | TEXT                | 学院描述 |

users (用户表)

| 字段         | 类型                 | 说明                                        |
|------------|--------------------|-------------------------------------------|
| id         | INTEGER PK         | 自增主键                                      |
| username   | VARCHAR(50) UNIQUE | 登录用户名                                     |
| password   | VARCHAR(255)       | 密码 (明文)                                   |
| role       | VARCHAR(20)        | admin / college_admin / teacher / student |
| name       | VARCHAR(50)        | 显示姓名                                      |
| college_id | INTEGER FK         | 所属学院 (仅 teacher/student)                  |
| email      | VARCHAR(100)       | 邮箱 (可选)                                   |
| phone      | VARCHAR(20)        | 电话 (可选)                                   |

courses (课程表)

| 字段                       | 类型           | 说明                |
|--------------------------|--------------|-------------------|
| id                       | INTEGER PK   | 自增主键              |
| name                     | VARCHAR(100) | 课程名称              |
| description              | TEXT         | 课程描述              |
| credits                  | REAL         | 学分                |
| teacher_id               | INTEGER FK   | 授课教师              |
| college_id               | INTEGER FK   | 所属学院              |
| max_students             | INTEGER      | 限选人数              |
| prerequisites            | TEXT         | 先修课程名称列表 (JSON数组) |
| syllabus                 | TEXT         | 课程介绍/大纲           |
| UNIQUE(teacher_id, name) |              | 同一教师不能有同名课程       |

course\_selections (选课记录表)

| 字段                            | 类型          | 说明                                         |
|-------------------------------|-------------|--------------------------------------------|
| id                            | INTEGER PK  | 自增主键                                       |
| student_id                    | INTEGER FK  | 学生                                         |
| course_id                     | INTEGER FK  | 课程                                         |
| status                        | VARCHAR(20) | pending=待抽签 / confirmed=中签 / cancelled=未中签 |
| UNIQUE(student_id, course_id) |             | 同一学生不能重复选同一门课                              |

grades (成绩表)

| 字段                            | 类型         | 说明            |
|-------------------------------|------------|---------------|
| id                            | INTEGER PK | 自增主键          |
| student_id                    | INTEGER FK | 学生            |
| course_id                     | INTEGER FK | 课程            |
| score                         | REAL       | 分数 (0-100)    |
| grade_point                   | REAL       | 绩点 (自动计算)     |
| recorded_by                   | INTEGER FK | 录入教师          |
| UNIQUE(student_id, course_id) |            | 同学生同一课程只有一条成绩 |

messages (消息表)

| 字段          | 类型         | 说明           |
|-------------|------------|--------------|
| id          | INTEGER PK | 自增主键         |
| sender_id   | INTEGER FK | 发送人          |
| receiver_id | INTEGER FK | 接收人          |
| course_id   | INTEGER FK | 关联课程 (可选)    |
| content     | TEXT       | 消息内容         |
| reply_to    | INTEGER FK | 回复的消息ID (可选) |
| is_read     | INTEGER    | 是否已读 (0/1)   |

course\_materials (课件表)

| 字段 | 类型         | 说明   |
|----|------------|------|
| id | INTEGER PK | 自增主键 |

| 字段          | 类型           | 说明       |
|-------------|--------------|----------|
| course_id   | INTEGER FK   | 所属课程     |
| filename    | VARCHAR(255) | 文件名      |
| file_path   | VARCHAR(500) | 存储路径     |
| file_size   | INTEGER      | 文件大小（字节） |
| uploaded_by | INTEGER FK   | 上传者      |

## 5.2 绩点计算规则

| 分数     | 绩点  |
|--------|-----|
| 90-100 | 4.0 |
| 80-89  | 3.0 |
| 70-79  | 2.0 |
| 60-69  | 1.0 |
| 0-59   | 0.0 |

## 5.3 抽签规则

- 当课程的状态为 `status='pending'` 选课人数  $\leq$  `max_students` 时，全部自动中签
- 当超过限选人数时，随机抽取 `max_students` 人，其余标记为 `cancelled`
- 抽签由学院管理员操作

# 6. 通用开发模式

## 6.1 后端路由权限校验

每个路由文件都定义了一个权限校验函数，返回 `(user_id, None, None)` 表示通过，`(None, error_json, 403)` 表示拒绝：

```
def admin_required():
    user = session.get('user')
    if not user or user.get('role') != 'admin':
        return None, jsonify({'success': False, 'message': '无权限'}), 403
    return user.get('id'), None, None

@admin_bp.route('/students', methods=['GET'])
def list_students():
    admin_id, err, resp = admin_required()
    if err:
        return resp
    # ... 业务逻辑
```

## 6.2 后端数据库写操作模板

所有写操作必须包含表间校验 + try-except rollback:

```
data = request.get_json()
teacher_id = data.get('teacher_id')

# 1. 校验外键
teacher = User.find_by_id(teacher_id)
if not teacher or teacher.role != 'teacher':
    return jsonify({'success': False, 'message': '教师不存在'})

# 2. 保存并异常回滚
try:
    course = Course()
    course.name = data['name']
    course.teacher_id = teacher_id
    course.college_id = teacher.college_id
    course.save()
    return jsonify({'success': True, 'message': '创建成功'})
except Exception as e:
    get_db().rollback()
    return jsonify({'success': False, 'message': f'保存失败: {str(e)}'})
```

## 6.3 前端 API 调用

所有接口封装在 `api.js` 中, 按角色分组:

```
// 认证
const res = await api.auth.login(username, password);

// 管理员
const res = await api.admin.listStudents({ page: 1, keyword: '张三' });

// 学院管理员
const res = await api.collegeAdmin.listTeachers({ page: 1 });

// 教师
const res = await api.teacher.listMyCourses();

// 学生
const res = await api.student.selectCourse(courseId);
```

## 6.4 前端模态框模式

```
// 显示表单模态框
document.getElementById('formModalTitle').textContent = '标题';
document.getElementById('formModalBody').innerHTML = `表单HTML`;
const modal = new bootstrap.Modal(document.getElementById('formModal'));

// 绑定保存按钮
document.getElementById('btnFormSave').onclick = async function() {
    const data = { /* 从表单取值 */ };
    // ... 后续逻辑 ...
}
```

```
const res = await api.xxx.save(data);
if (res.success) {
  modal.hide();
  refreshTable(); // 刷新列表
}
showAlert(res.message);
};
modal.show();
```

## 6.5 前端 CSV 下载模式

```
const res = await fetch('/api/...');
const blob = await res.blob();
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = '文件名.csv';
a.click();
URL.revokeObjectURL(url);
```

## 6.6 事件委托（动态DOM）

页面上动态生成的按钮不直接绑定 onclick，而是在父容器上用事件委托：

```
document.getElementById('courseCards').addEventListener('click', function(e) {
  const btn = e.target.closest('.btn-edit-course');
  if (!btn) return;
  const course = JSON.parse(btn.dataset.course);
  showEditForm(course);
});
```

# 7. 系统管理员 (admin)

## 7.1 页面文件

frontend/pages/admin.html — 约 820 行

## 7.2 功能列表

| 功能   | 说明                     |
|------|------------------------|
| 控制台  | 统计学生数、教师数、课程数、学院数      |
| 学生管理 | 增删改查，按学号/姓名搜索，按学院筛选    |
| 教师管理 | 增删改查，按工号/姓名搜索，按学院筛选    |
| 课程管理 | 增删改查，按名称搜索，按学院筛选，教师选下拉 |
| 成绩管理 | 增删改查，按学生ID筛选           |
| 学院管理 | 新建学院（自动生成学院管理员账号），查看列表 |

| 功能   | 说明                 |
|------|--------------------|
| 选课统计 | 柱状图展示各课程选课人数，导出CSV |

## 7.3 关键后端路由

backend/routes/admin.py

- GET /api/admin/students — 支持 ?page=&page\_size=&college\_id=&keyword=
- POST /api/admin/students — 创建学生，password 必填
- POST /api/admin/colleges — 创建学院，自动生成 {学院名}\_admin 的 college\_admin 账号

## 7.4 选课统计接口

```
# 获取统计数据
GET /api/admin/statistics/course-selection
# 返回: [{course_id, course_name, count}, ...]

# 导出CSV
GET /api/admin/statistics/course-selection/export
# 返回: CSV文件下载, 列: 课程ID, 课程名称, 选课人数
```

# 8. 学院管理员 (college\_admin)

## 8.1 页面文件

frontend/pages/college\_admin.html — 约 1100 行

## 8.2 功能列表

| 功能   | 说明                         |
|------|----------------------------|
| 控制台  | 统计本学院教师、学生、课程数             |
| 教师管理 | 本学院教师的增删改查，支持关键字搜索         |
| 学生管理 | 本学院学生的增删改查，支持关键字搜索         |
| 课程管理 | 本学院课程的增删改查，支持关键字搜索         |
| 选课管理 | 查看本学院所有选课记录                |
| 抽签管理 | 选择课程 → 执行抽签 → 查看结果 → 导出CSV |
| 成绩管理 | 本学院课程成绩的增删改查               |

## 8.3 关键后端路由

backend/routes/college\_admin.py

- POST /api/college-admin/lottery/<course\_id> — 执行抽签
  - 请求体: {"max\_students": 30}



- 返回中签学生列表
- `GET /api/college-admin/lottery/<course_id>/result` — 查看抽签结果
- `GET /api/college-admin/lottery/<course_id>/export` — 导出中签名单CSV

## 8.4 抽签流程

1. 学院管理员进入"抽签管理"页面
2. 下拉选择课程（只显示本学院课程）
3. 可调整限选人数
4. 点击"执行抽签"
5. 查看中签和未中签学生名单
6. 可导出中签名单CSV

## 8.5 成绩CRUD

学院管理员可以跨课程管理成绩：

- 录入：选择学生（本学院）和课程（本学院），输入分数
- 绩点按规则自动计算，实时预览
- 编辑/删除已有的成绩记录

# 9. 教师 (teacher)

## 9.1 页面文件

`frontend/pages/teacher.html` — 约 730 行

## 9.2 功能列表

| 功能   | 说明                                    |
|------|---------------------------------------|
| 控制台  | 统计我的课程数、中签学生总数、未读消息                   |
| 我的课程 | 查看课程列表，编辑课程信息，上传/查看/删除课件              |
| 中签学生 | 选择课程 → 查看中签学生名单                       |
| 成绩录入 | 选择课程 → 录入/修改分数（自动计算绩点、实时预览）           |
| 成绩统计 | 选择课程 → 分数段分布表 + 柱状图 + 平均分/最高分/最低分/及格率 |
| 消息   | 收件箱/发件箱/发送消息（选择课程→选择学生→发送）            |

## 9.3 关键后端路由

`backend/routes/teacher.py`

课程信息管理：

- `GET /api/teacher/courses` — 我的课程列表（含选课人数）
- `PUT /api/teacher/courses/<id>` — 更新课程信息（描述、学分、限选、先修课程、大纲）

课件管理:

- `POST /api/teacher/courses/<id>/upload-material` — 上传课件 (支持多文件, `files` 字段)
- `GET /api/teacher/courses/<id>/materials` — 课件列表
- `DELETE /api/teacher/materials/<id>` — 删除课件 (会删除文件)

成绩管理:

- `GET /api/teacher/courses/<id>/grades` — 查看已录入成绩
- `POST /api/teacher/courses/<id>/grades` — 批量保存成绩
  - 请求体: `{"grades": [{"student_id": 14, "score": 92}, ...]}`
- `GET /api/teacher/courses/<id>/statistics` — 成绩统计
  - 返回: `{total, avg_score, max_score, min_score, pass_rate, score_distribution}`
  - `score_distribution` 格式: `{"0-59": 0, "60-69": 1, "70-79": 0, "80-89": 0, "90-100": 2}`

消息:

- `POST /api/teacher/messages/send` — 发送消息
  - 请求体: `{"receiver_id": 14, "course_id": 2, "content": "请提交作业"}`
- `GET /api/teacher/messages/received?page=1&page_size=20` — 收件箱
- `GET /api/teacher/messages/sent?page=1&page_size=20` — 发件箱

9.4 成绩录入注意事项

- 只有已中签的学生才会出现在成绩录入表中
- 已录入的成绩会自动回填显示
- 绩点根据分数实时计算并在界面上预览
- 保存时使用 `DELETE + INSERT` 策略, 确保同一个学生同一课程只有一条成绩记录

10. 学生 (student)

10.1 页面文件

`frontend/pages/student.html` — 约 520 行

10.2 功能列表

| 功能   | 说明                        |
|------|---------------------------|
| 控制台  | 统计可选课程数、已选课程数、已获学分        |
| 个人信息 | 查看和修改邮箱、电话                |
| 选课   | 浏览课程列表 (可按学院筛选), 查看详情, 选课 |
| 已选课程 | 查看选课状态 (待抽签/中签/未中签), 可取消  |

| 功能   | 说明                        |
|------|---------------------------|
| 我的课程 | 中签的课程列表，显示课件数量，可查看详情和下载课件 |
| 成绩查询 | 查看各课程成绩和绩点                |
| 消息   | 收件箱/发件箱/发送消息/回复消息         |

## 10.3 关键后端路由

backend/routes/student.py

### 课程浏览与选课：

- GET /api/student/courses?page=1&page\_size=20&college\_id= — 课程列表
- POST /api/student/select-course — 选课
  - 请求体：{"course\_id": 2}
  - **自动检查先修课程**：查询学生是否修过同名课程且分数 $\geq 60$ ，未修过则拒绝
  - 检查是否已选过该课程
  - 选课状态初始为 pending

### 选课管理：

- GET /api/student/my-selections — 我的选课记录（含状态和课程信息）
- POST /api/student/cancel-selection/<id> — 取消选课

### 课件下载（权限控制）：

- GET /api/student/courses/<id>/materials — 课件列表（仅中签学生可查看）
- GET /api/student/courses/<id>/materials/<id>/download — 下载课件（仅中签学生可下载）

### 成绩：

- GET /api/student/my-grades — 我的成绩（含课程名称、学分）

### 消息：

- POST /api/student/messages/send — 发送消息给教师
- POST /api/student/messages/<id>/reply — 回复消息（自动关联原消息）
- GET /api/student/messages/received — 收件箱
- GET /api/student/messages/sent — 发件箱

## 10.4 先修课程检查逻辑

1. 课程表中 prerequisites 字段存储的是先修课程**名称**列表（JSON数组），如 ["C语言程序设计"]
2. 选课时，后端查询该学生 score  $\geq 60$  的成绩记录中对应的课程名称
3. 将先修课程名称与已通过课程名称做匹配（同名即视为满足）
4. 有一门不满足则返回错误提示，拒绝选课

## 11. 测试账号

### 系统管理员

| 用户名   | 密码       | 说明    |
|-------|----------|-------|
| admin | admin123 | 系统管理员 |

### 学院管理员

| 用户名            | 密码       | 所属学院    |
|----------------|----------|---------|
| computer_admin | admin123 | 计算机学院   |
| math_admin     | admin123 | 数学与统计学院 |
| software_admin | admin123 | 软件学院    |
| ai_admin       | admin123 | 人工智能学院  |

### 教师

| 用户名      | 密码     | 所属学院    |
|----------|--------|---------|
| teacher1 | 123456 | 计算机学院   |
| teacher2 | 123456 | 计算机学院   |
| teacher3 | 123456 | 数学与统计学院 |
| teacher4 | 123456 | 数学与统计学院 |
| teacher5 | 123456 | 软件学院    |
| teacher6 | 123456 | 软件学院    |
| teacher7 | 123456 | 人工智能学院  |
| teacher8 | 123456 | 人工智能学院  |

### 学生

| 用户名      | 密码     | 所属学院    |
|----------|--------|---------|
| student1 | 123456 | 计算机学院   |
| student2 | 123456 | 计算机学院   |
| student3 | 123456 | 计算机学院   |
| student4 | 123456 | 数学与统计学院 |
| student5 | 123456 | 数学与统计学院 |
| student6 | 123456 | 数学与统计学院 |

| 用户名       | 密码     | 所属学院   |
|-----------|--------|--------|
| student7  | 123456 | 软件学院   |
| student8  | 123456 | 软件学院   |
| student9  | 123456 | 软件学院   |
| student10 | 123456 | 人工智能学院 |
| student11 | 123456 | 人工智能学院 |
| student12 | 123456 | 人工智能学院 |

## 12. 常见问题

### Q: 数据库被锁定?

- 所有写操作已实现 `try-except + get_db().rollback()` 回滚，如果遇到锁请重启后端
- SQLite WAL 模式支持并发读，但不支持并发写

### Q: 重置数据库?

删除 `database/course_selection.db` (以及 `.db-shm`、`.db-wal` 文件)，重启后端即可自动重建

### Q: 上传的课件存放在哪里?

`backend/uploads/course_{课程ID}/` 目录下，可通过 `/uploads/course_{课程ID}/{文件名}` 访问

### Q: 密码修改?

目前密码修改功能未实现 (后端路由标记为 TODO)

### Q: 学生看不到课件?

学生必须**中签**该课程后才能查看和下载课件，未中签课程看不到课件列表